

Desarrollo de software Backend

Inv p1 U3

Variables

POO

Carlos Leonel

YIRA BAILON

3.1- Variables:

Locales: Se definen dentro de una función o método y solo existen allí.

Globales: Se definen a nivel de script y se pueden acceder desde cualquier parte.

De objeto (Instancia): Pertenecen a una instancia específica. Se definen usando self.

Clase: Compartidas por todas las instancias de una clase. Se definen directamente en el cuerpo de la clase.

```
variable_global = "Soy global"

class Persona:
    variable_clase = "Humano" # Compartida por todos

    def __init__(self, nombre):
        self.variable_objeto = nombre # Única para cada objeto

    def caminar(self):
        variable_local = "Caminando a 5 km/h" # Solo existe en este método
        print(f"{self.variable_objeto} está {variable_local}")

# Ejemplo de uso
p1 = Persona("Carlos")
p2 = Persona("Ana")

print(p1.variable_objeto) # Carlos
print(p2.variable_objeto) # Ana
print(Persona.variable_clase) # Humano
```

3.1.3 Sobrecargando Métodos (Method Overloading)

A diferencia de lenguajes como Java o C++, Python **no soporta la sobrecarga de métodos tradicional** (definir varios métodos con el mismo nombre pero diferentes argumentos en la misma clase). Si defines dos métodos con el mismo nombre, el último reemplaza al primero.

Sin embargo, en Python "simulamos" la sobrecarga usando **argumentos por defecto** o argumentos variables (*args, kwargs).

```
class Calculadora:
    # Simulamos sobrecarga definiendo argumentos opcionales (None)
    def sumar(self, a, b, c = None):
        if c is not None:
            return a + b + c
        return a + b

calc = Calculadora()
print(calc.sumar(5, 10))      # Funciona con 2 parámetros (15)
print(calc.sumar(5, 10, 2))  # Funciona con 3 parámetros (17)
```

3.1.4 Encapsulación

Es el ocultamiento de los datos internos de un objeto para protegerlos de modificaciones no autorizadas. En Python no existen las palabras clave `private` o `public`, sino que usamos una **convención de guiones bajos**:

variable: `"Protected"` (Aviso para otros desarrolladores de que no deben tocarla fuera de la clase).

variable: `"Private"` (Activa el *Name Mangling*, haciendo que Python cambie el nombre internamente para que sea difícil acceder a ella directamente).

```
class CuentaBancaria:
    def __init__(self, titular, saldo_inicial):
        self.titular = titular
        self.__saldo = saldo_inicial # Atributo privado

    # Getter
    @property
    def saldo(self):
        return self.__saldo

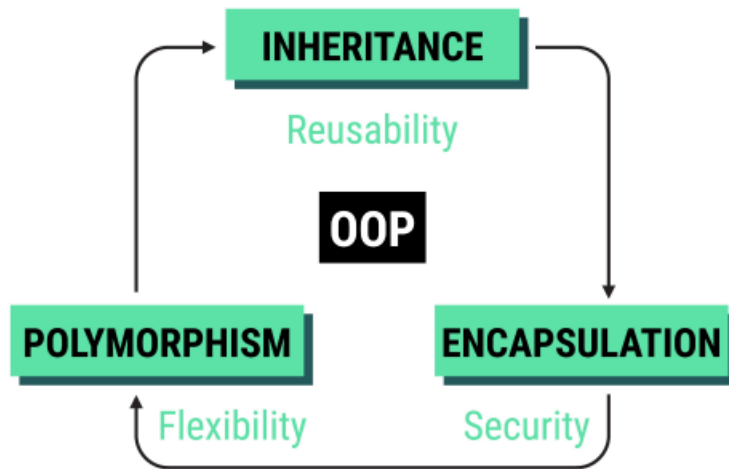
    # Setter
    @saldo.setter
    def saldo(self, cantidad):
        if cantidad >= 0:
            self.__saldo = cantidad
        else:
            print("¡Error: El saldo no puede ser negativo!")

cuenta = CuentaBancaria("Sofía", 1000)
# print(cuenta.__saldo) # Lanza un AttributeError

print(cuenta.saldo) # Acceso mediante el getter: 1000
cuenta.saldo = 1500 # Modificación mediante el setter
```

3.1.5 Polimorfismo

El polimorfismo significa "muchas formas". Permite que diferentes clases tengan métodos con el mismo nombre pero con comportamientos distintos. Python utiliza *Duck Typing* ("Si camina como pato y grazna como pato, entonces es un pato").



Ejemplo:

```
class Gato:
    def hacer_sonido(self):
        return "Miau"

class Perro:
    def hacer_sonido(self):
        return "Guau"

# Una función que recibe cualquier objeto que tenga el método 'hacer_sonido'
def escuchar_animal(animal):
    print(animal.hacer_sonido())

gato = Gato()
perro = Perro()

escuchar_animal(gato) # Miau
escuchar_animal(perro) # Guau
```

3.2 Clases Anidadas (Nested Classes)

Una clase anidada es simplemente una clase definida dentro de otra clase. Se utiliza cuando una clase secundaria no tiene sentido de existir sin la clase principal que la contiene (relación de fuerte dependencia).

```
class Smartphone:
    def __init__(self, marca, modelo, ram_gb):
        self.marca = marca
        self.modelo = modelo
        # Instanciamos la clase anidada dentro de la principal
        self.procesador = self.Procesador("Snapdragon 8 Gen 3", 8)

    def mostrar_especificaciones(self):
        print(f"Teléfono: {self.marca} {self.modelo}")
        # Accedemos a los métodos de la clase interna
        print(f"Procesador: {self.procesador.obtener_info_cpu()}")

# Clase Anidada
class Procesador:
    def __init__(self, modelo_cpu, nucleos):
        self.modelo_cpu = modelo_cpu
        self.nucleos = nucleos

    def obtener_info_cpu(self):
        return f"{self.modelo_cpu} de {self.nucleos} núcleos"

# Uso
mi_telefono = Smartphone("Samsung", "Galaxy S24", 12)
mi_telefono.mostrar_especificaciones()
```

Referencias

- <https://docs.python.org/es/3/tutorial/classes.html>
- <https://realpython.com/ref/glossary/duck-typing/>